Open in Colab

42 Days of AI/ML Challenge

Prerequisites Checklist

Before you start Day 1, work through this notebook.

This is not optional. Every concept in the 42-day challenge builds on what's here. If you struggle with any section, go back to the resources listed in that section before moving forward.

The notebook covers 4 areas:

1. Python for ML
2. Linear Algebra
3. Calculus
4. Statistics and Probability

At the end, there's a **Final Self-Check**: 5 exercises that tell you whether you're ready for Day 1.

How to use this notebook:

- Read the concept explanation
- Run the example code
- Complete the exercise (try it yourself before looking at the solution)
- If an exercise feels very difficult, spend more time on that section's resources

Estimated time: 3-5 days if you know Python. 3-4 weeks if starting from scratch.

In [ ]:
```
# Run this first. Installs everything you need for this notebook.
!pip install numpy pandas matplotlib scipy scikit-learn -q
print('All dependencies installed.')
```

Part 1: Python for ML

Resources if this section feels unfamiliar:

- CS50P by Harvard: cs50.harvard.edu/python (best free Python course)
- NumPy official docs: numpy.org/learn

- Pandas getting started: pandas.pydata.org/docs/getting_started

You need to be comfortable with Python functions, loops, list comprehensions, and the 3 core ML libraries: NumPy, Pandas, and Matplotlib.

1.1 Python Fundamentals

In [ ]:
```python
# CONCEPT: Functions, loops, list comprehensions
# These are the building blocks of every ML preprocessing script you will write.

# Functions
def compute_stats(numbers):
    mean = sum(numbers) / len(numbers)
    sorted_nums = sorted(numbers)
    n = len(sorted_nums)
    median = sorted_nums[n//2] if n % 2 != 0 else (sorted_nums[n//2-1] + sorted_num
    return {'mean': mean, 'median': median}

sample = [12, 45, 7, 23, 56, 89, 34, 12, 67, 45]
print('Stats:', compute_stats(sample))

# List comprehensions (used constantly in ML preprocessing)
squared = [x**2 for x in sample]
above_mean = [x for x in sample if x > compute_stats(sample)['mean']]
print('Squared:', squared)
print('Above mean:', above_mean)
```

In [ ]:
```python
# EXERCISE 1.1
# Write a function that takes a list of numbers and returns:
# - mean
# - median
# - the count of values above the mean
# - the count of values below the mean
# Use only base Python. No NumPy yet.

def analyse_numbers(numbers):
    # your code here
    pass

test_data = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
print(analyse_numbers(test_data))
# Expected: {'mean': 55.0, 'median': 55.0, 'above_mean': 5, 'below_mean': 5}
```

In [ ]:
```python
# SOLUTION 1.1
def analyse_numbers(numbers):
    n = len(numbers)
    mean = sum(numbers) / n
    sorted_nums = sorted(numbers)
    median = sorted_nums[n//2] if n % 2 != 0 else (sorted_nums[n//2-1] + sorted_num
    above_mean = len([x for x in numbers if x > mean])
    below_mean = len([x for x in numbers if x < mean])
    return {'mean': mean, 'median': median, 'above_mean': above_mean, 'below_mean':
```

```
test_data = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
print(analyse_numbers(test_data))
```

1.2 NumPy

NumPy is how ML libraries store and operate on data internally. Every time you call model.fit(X, y), X is a NumPy array underneath. Understanding arrays, shapes, and operations prevents silent bugs in preprocessing.

In [ ]:
```
import numpy as np

# CONCEPT: Arrays, shapes, operations
arr = np.array([1, 2, 3, 4, 5, 6])
matrix = arr.reshape(2, 3)

print('Array:', arr)
print('Shape:', arr.shape)
print('Matrix:\n', matrix)
print('Matrix shape:', matrix.shape)

# Broadcasting: NumPy applies operations element-wise
print('\nArray * 2:', arr * 2)
print('Array + 10:', arr + 10)
print('Array mean:', arr.mean())
print('Array std:', arr.std())

# Matrix operations used in ML
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print('\nMatrix multiplication (dot product):\n', np.dot(A, B))
print('Transpose of A:\n', A.T)
```

In [ ]:
```
# EXERCISE 1.2
# Create a 4x4 NumPy matrix of random integers between 1 and 100.
# Then compute:
# - Mean of the entire matrix
# - Mean of each row
# - Mean of each column
# - The transpose
# - Dot product of the matrix with its transpose

np.random.seed(42)
# your code here
```

In [ ]:
```
# SOLUTION 1.2
np.random.seed(42)
M = np.random.randint(1, 100, size=(4, 4))
print('Matrix:\n', M)
print('Overall mean:', M.mean())
print('Row means:', M.mean(axis=1))
print('Column means:', M.mean(axis=0))
```

```
print('Transpose:\n', M.T)
print('Dot product with transpose:\n', np.dot(M, M.T))
```

1.3 Pandas

Pandas is how you load, inspect, and clean real datasets before modelling. Every ML project starts here. You need to be comfortable loading data, checking for issues, filtering, grouping, and handling missing values.

In [ ]:
```
import pandas as pd
import numpy as np

# CONCEPT: DataFrame operations
# Create a sample dataset similar to what you'll see in Week 1
np.random.seed(42)
df = pd.DataFrame({
    'age': np.random.randint(22, 60, 100),
    'salary': np.random.randint(30000, 150000, 100),
    'department': np.random.choice(['Engineering', 'Marketing', 'Sales', 'HR'], 100
    'experience_years': np.random.randint(0, 20, 100),
    'churned': np.random.choice([0, 1], 100, p=[0.8, 0.2])
})

# Introduce some missing values
df.loc[np.random.choice(df.index, 15), 'salary'] = np.nan

print('Shape:', df.shape)
print('\nFirst 5 rows:')
print(df.head())
print('\nData types:')
print(df.dtypes)
print('\nMissing values:')
print(df.isnull().sum())
print('\nBasic stats:')
print(df.describe())
```

In [ ]:
```
# EXERCISE 1.3
# Using the df created above, do the following:
# 1. Find the average salary per department
# 2. Find the churn rate per department (mean of 'churned' column per department)
# 3. Fill missing salary values with the median salary of that employee's departmen
# 4. Create a new column 'senior' that is 1 if experience_years >= 10, else 0
# 5. Filter the dataframe to show only senior employees who churned

# your code here
```

In [ ]:
```
# SOLUTION 1.3

# 1. Average salary per department
print('Average salary per department:')
print(df.groupby('department')['salary'].mean())

# 2. Churn rate per department
```

```
print('\nChurn rate per department:')
print(df.groupby('department')['churned'].mean())

# 3. Fill missing salary with department median
df['salary'] = df.groupby('department')['salary'].transform(lambda x: x.fillna(x.me
print('\nMissing values after fill:', df['salary'].isnull().sum())

# 4. Create senior column
df['senior'] = (df['experience_years'] >= 10).astype(int)

# 5. Filter senior employees who churned
senior_churned = df[(df['senior'] == 1) & (df['churned'] == 1)]
print('\nSenior employees who churned:')
print(senior_churned[['age', 'salary', 'department', 'experience_years']].head())
```

1.4 Matplotlib

You will plot data every single day in this challenge. EDA, distributions, model performance, learning curves. Matplotlib is the foundation. Know how to build histograms, scatter plots, and subplots without googling the syntax.

In [ ]:
```
import matplotlib.pyplot as plt
import numpy as np

# CONCEPT: Core plot types used in ML
np.random.seed(42)
data = np.random.normal(loc=50, scale=15, size=500)

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Histogram: used for distribution analysis
axes[0].hist(data, bins=30, color='steelblue', edgecolor='white')
axes[0].set_title('Histogram')
axes[0].set_xlabel('Value')
axes[0].set_ylabel('Frequency')

# Scatter plot: used for relationship analysis
x = np.random.randn(100)
y = 2 * x + np.random.randn(100) * 0.5
axes[1].scatter(x, y, alpha=0.6, color='steelblue')
axes[1].set_title('Scatter Plot')
axes[1].set_xlabel('Feature X')
axes[1].set_ylabel('Target Y')

# Box plot: used for outlier detection
axes[2].boxplot(data)
axes[2].set_title('Box Plot')
axes[2].set_ylabel('Value')

plt.tight_layout()
plt.show()
```

In [ ]:
```python
# EXERCISE 1.4
# Using the employee df from Exercise 1.3:
# Create a 2x2 subplot figure with:
# - Top left: Histogram of age
# - Top right: Histogram of salary
# - Bottom left: Scatter plot of experience_years vs salary
# - Bottom right: Bar chart of average salary per department
# Add titles and axis labels to every subplot.

# your code here
```

In [ ]:
```python
# SOLUTION 1.4
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

axes[0, 0].hist(df['age'], bins=20, color='steelblue', edgecolor='white')
axes[0, 0].set_title('Age Distribution')
axes[0, 0].set_xlabel('Age')
axes[0, 0].set_ylabel('Count')

axes[0, 1].hist(df['salary'], bins=20, color='steelblue', edgecolor='white')
axes[0, 1].set_title('Salary Distribution')
axes[0, 1].set_xlabel('Salary')
axes[0, 1].set_ylabel('Count')

axes[1, 0].scatter(df['experience_years'], df['salary'], alpha=0.5, color='steelblu
axes[1, 0].set_title('Experience vs Salary')
axes[1, 0].set_xlabel('Experience (years)')
axes[1, 0].set_ylabel('Salary')

dept_salary = df.groupby('department')['salary'].mean()
axes[1, 1].bar(dept_salary.index, dept_salary.values, color='steelblue')
axes[1, 1].set_title('Average Salary by Department')
axes[1, 1].set_xlabel('Department')
axes[1, 1].set_ylabel('Average Salary')

plt.tight_layout()
plt.show()
```

Part 2: Linear Algebra

Resources if this section feels unfamiliar:

- 3Blue1Brown Essence of Linear Algebra: youtube.com/3blue1brown (watch all 16 videos)
- Mathematics for Machine Learning - Linear Algebra by Imperial College London (free on Coursera)

Linear algebra is the language ML algorithms are written in. Matrix multiplication is how neural networks pass data forward. Dot products measure similarity. Eigenvectors are the

core of PCA. You don't need to derive proofs. You need to understand what these operations do and implement them.

In [ ]:
```python
import numpy as np

# CONCEPT: Vectors and dot products
# In ML, features are vectors. Measuring similarity between two data points
# is a dot product. This is what recommendation systems do at their core.

user_A = np.array([1, 0, 1, 1, 0])  # User A's movie preferences (1=liked)
user_B = np.array([1, 1, 1, 0, 0])  # User B's movie preferences
user_C = np.array([0, 0, 0, 1, 1])  # User C's movie preferences

# Dot product as similarity
sim_AB = np.dot(user_A, user_B)
sim_AC = np.dot(user_A, user_C)

print('Similarity A-B (dot product):', sim_AB)  # Higher = more similar
print('Similarity A-C (dot product):', sim_AC)
print('User A is more similar to User', 'B' if sim_AB > sim_AC else 'C')
```

In [ ]:
```python
# CONCEPT: Matrix multiplication
# Every forward pass in a neural network is a series of matrix multiplications.
# Input data (matrix) multiplied by weights (matrix) = output (matrix).

# Simulating one layer of a neural network
np.random.seed(42)
X = np.random.randn(5, 3)   # 5 samples, 3 features each
W = np.random.randn(3, 4)   # Weight matrix: 3 inputs, 4 neurons
b = np.random.randn(4)      # Bias for each neuron

output = np.dot(X, W) + b   # Forward pass
print('Input shape:', X.shape)
print('Weight shape:', W.shape)
print('Output shape:', output.shape)
print('Output (pre-activation):\n', output.round(3))
```

In [ ]:
```python
# CONCEPT: Eigenvalues and Eigenvectors
# PCA (Day 23) uses eigenvectors to find the directions of maximum variance in data
# This is what allows you to reduce 100 features to 10 while keeping most informati

np.random.seed(42)
# Create correlated data (like real feature data)
mean = [0, 0]
cov = [[3, 2], [2, 2]]  # Covariance matrix
data = np.random.multivariate_normal(mean, cov, 200)

# Compute covariance matrix
cov_matrix = np.cov(data.T)
print('Covariance matrix:\n', cov_matrix.round(3))

# Compute eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
print('\nEigenvalues:', eigenvalues.round(3))
```

```
print('Eigenvectors:\n', eigenvectors.round(3))
print('\nVariance explained by each component:', (eigenvalues / eigenvalues.sum() *
```

In [ ]:
```
# EXERCISE 2.1
# You have a dataset of 3 users and their ratings for 4 movies (0 = not watched, 1-
# User 1: [5, 3, 0, 1]
# User 2: [4, 0, 4, 1]
# User 3: [1, 1, 0, 5]

# 1. Create this as a NumPy matrix
# 2. Compute the dot product of User 1 and User 2 rating vectors
# 3. Compute the dot product of User 1 and User 3 rating vectors
# 4. Based on dot product similarity, which user is more similar to User 1?
# 5. Compute the transpose of the matrix and explain what it represents

# your code here
```

In [ ]:
```
# SOLUTION 2.1
ratings = np.array([
    [5, 3, 0, 1],
    [4, 0, 4, 1],
    [1, 1, 0, 5]
])

sim_1_2 = np.dot(ratings[0], ratings[1])
sim_1_3 = np.dot(ratings[0], ratings[2])

print('Dot product User1-User2:', sim_1_2)
print('Dot product User1-User3:', sim_1_3)
print('User 1 is more similar to User', '2' if sim_1_2 > sim_1_3 else '3')
print('\nTranspose (movies x users - each row is now a movie, each column a user):'
print(ratings.T)
```

Part 3: Calculus

Resources if this section feels unfamiliar:

- 3Blue1Brown Essence of Calculus: youtube.com/3blue1brown (chapters 1-8)
- Khan Academy Calculus 1: khanacademy.org/math/calculus-1 (derivatives section)

You need derivatives and the chain rule. That's it. Gradient descent, which is how every ML model learns, is just repeatedly computing derivatives and moving in the direction that reduces error. If you understand that, you understand 80% of how models train.

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt

# CONCEPT: Derivatives and Gradient Descent
# A derivative tells you the slope of a function at any point.
```

```
# Gradient descent uses derivatives to find the minimum of a loss function.
# This is exactly what happens when you call model.fit().

# Simple loss function: L(w) = w^2 (minimum at w=0)
def loss(w):
    return w ** 2

def gradient(w):
    return 2 * w  # Derivative of w^2

# Gradient descent
w = 10.0       # Start far from minimum
lr = 0.1       # Learning rate
history = [w]

for step in range(30):
    grad = gradient(w)
    w = w - lr * grad  # Move opposite to gradient
    history.append(w)

# Plot convergence
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
w_range = np.linspace(-12, 12, 100)
plt.plot(w_range, loss(w_range), 'b-', label='Loss function')
plt.scatter(history[0], loss(history[0]), color='red', s=100, zorder=5, label='Star
plt.scatter(history[-1], loss(history[-1]), color='green', s=100, zorder=5, label='
plt.title('Loss Function L(w) = w²')
plt.xlabel('w')
plt.ylabel('Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(len(history)), [loss(w) for w in history], 'b-o', markersize=4)
plt.title('Loss Over Training Steps')
plt.xlabel('Step')
plt.ylabel('Loss')

plt.tight_layout()
plt.show()
print(f'Final w: {history[-1]:.6f} (should be close to 0)')
print(f'Final loss: {loss(history[-1]):.6f}')
```

In [ ]:

```
# CONCEPT: Partial Derivatives
# Real ML models have thousands of parameters (weights).
# You compute the partial derivative of the loss with respect to each weight.
# That tells you how much each weight contributed to the error.

# Loss function with 2 parameters: L(w1, w2) = w1^2 + w2^2
def loss_2d(w1, w2):
    return w1**2 + w2**2

def partial_w1(w1, w2):
    return 2 * w1  # dL/dw1
```

```
def partial_w2(w1, w2):
    return 2 * w2  # dL/dw2

# Gradient descent with 2 parameters
w1, w2 = 8.0, -6.0
lr = 0.1

print('Starting point: w1={:.2f}, w2={:.2f}, loss={:.2f}'.format(w1, w2, loss_2d(w1

for step in range(20):
    w1 = w1 - lr * partial_w1(w1, w2)
    w2 = w2 - lr * partial_w2(w1, w2)

print('After 20 steps: w1={:.4f}, w2={:.4f}, loss={:.4f}'.format(w1, w2, loss_2d(w1
```

In [ ]:
```
# EXERCISE 3.1
# Implement gradient descent for L(w) = (w - 3)^2
# This loss function has its minimum at w = 3.

# 1. Write the loss function
# 2. Write the gradient (derivative) of the loss function
# 3. Run gradient descent for 50 steps starting from w = -5
# 4. Use learning rate = 0.1
# 5. Print the final value of w (should be close to 3)
# 6. Plot the loss over training steps

# your code here
```

In [ ]:
```
# SOLUTION 3.1
def loss(w):
    return (w - 3) ** 2

def gradient(w):
    return 2 * (w - 3)

w = -5.0
lr = 0.1
loss_history = [loss(w)]

for step in range(50):
    w = w - lr * gradient(w)
    loss_history.append(loss(w))

print(f'Final w: {w:.6f} (should be close to 3)')
print(f'Final loss: {loss(w):.8f}')

plt.plot(loss_history)
plt.title('Loss Over Training Steps')
plt.xlabel('Step')
plt.ylabel('Loss')
plt.show()
```

Part 4: Statistics and Probability

Resources if this section feels unfamiliar:

- StatQuest with Josh Starmer: youtube.com/statquest (best stats resource for ML)
- Practical Statistics for Data Scientists - Peter Bruce (O'Reilly)

Statistics is the most underrated foundation in ML. It's how you understand your data before modelling, how you evaluate your model after training, and how you explain your model's behaviour to non-technical stakeholders.

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# CONCEPT: Distributions
# Most ML assumptions about data involve distributions.
# Linear regression assumes normally distributed residuals.
# Naive Bayes assumes features follow a distribution.
# Knowing what your data's distribution looks like is Step 1 of every ML project.

np.random.seed(42)
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# Normal distribution
normal_data = np.random.normal(loc=0, scale=1, size=1000)
axes[0].hist(normal_data, bins=40, density=True, color='steelblue', alpha=0.7)
x = np.linspace(-4, 4, 100)
axes[0].plot(x, stats.norm.pdf(x), 'r-', lw=2)
axes[0].set_title(f'Normal Distribution\nSkewness: {stats.skew(normal_data):.3f}')

# Right skewed (like income, house prices)
skewed_data = np.random.exponential(scale=2, size=1000)
axes[1].hist(skewed_data, bins=40, density=True, color='steelblue', alpha=0.7)
axes[1].set_title(f'Right Skewed (Exponential)\nSkewness: {stats.skew(skewed_data):

# Bimodal (like two customer segments)
bimodal_data = np.concatenate([np.random.normal(2, 0.5, 500), np.random.normal(6, 0
axes[2].hist(bimodal_data, bins=40, density=True, color='steelblue', alpha=0.7)
axes[2].set_title(f'Bimodal Distribution\nSkewness: {stats.skew(bimodal_data):.3f}'

plt.tight_layout()
plt.show()
```

In [ ]:
```
# CONCEPT: Probability and Bayes Theorem
# Bayes Theorem is the foundation of Naive Bayes classifiers and Bayesian ML.
# P(A|B) = P(B|A) * P(A) / P(B)
# In spam detection: P(spam | contains 'free') = ?

# Real example: Email spam detection
total_emails = 1000
spam_emails = 200
ham_emails = 800
```

```
# 'free' appears in 80% of spam, 10% of ham
p_free_given_spam = 0.80
p_free_given_ham = 0.10

p_spam = spam_emails / total_emails
p_ham = ham_emails / total_emails

# P(free) = P(free|spam)*P(spam) + P(free|ham)*P(ham)
p_free = p_free_given_spam * p_spam + p_free_given_ham * p_ham

# Bayes theorem: P(spam|free)
p_spam_given_free = (p_free_given_spam * p_spam) / p_free

print(f'P(spam): {p_spam:.2f}')
print(f'P(free | spam): {p_free_given_spam:.2f}')
print(f'P(free): {p_free:.2f}')
print(f'P(spam | free): {p_spam_given_free:.4f}')
print(f'\nIf an email contains the word "free", there is a {p_spam_given_free*100:.
```

In [ ]:
```
# CONCEPT: Correlation vs Causation
# Correlation measures the linear relationship between two variables (-1 to 1).
# High correlation does not mean one causes the other.
# This mistake causes more bad ML decisions than almost anything else.

np.random.seed(42)
n = 100

# Strong positive correlation
x = np.random.randn(n)
y_strong = 2 * x + np.random.randn(n) * 0.3

# Weak correlation
y_weak = 0.3 * x + np.random.randn(n) * 2

# No correlation
y_none = np.random.randn(n)

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, y, title in zip(axes,
                        [y_strong, y_weak, y_none],
                        ['Strong Correlation', 'Weak Correlation', 'No Correlation
    corr = np.corrcoef(x, y)[0, 1]
    ax.scatter(x, y, alpha=0.6, color='steelblue')
    ax.set_title(f'{title}\nr = {corr:.3f}')
    ax.set_xlabel('X')
    ax.set_ylabel('Y')

plt.tight_layout()
plt.show()
```

In [ ]:
```
# EXERCISE 4.1
# You have test scores for two groups of students:
# Group A (taught with new method): mean=75, std=10, n=50
```

```
# Group B (taught with old method): mean=70, std=12, n=50

# 1. Generate both groups using numpy random normal
# 2. Compute mean, std, skewness, and kurtosis for each group
# 3. Plot both distributions on the same histogram
# 4. Compute the correlation between the two groups
# 5. Run a t-test using scipy.stats.ttest_ind to check if
#    the difference is statistically significant (p < 0.05)
# 6. Print your conclusion in plain English

np.random.seed(42)
# your code here
```

In [ ]:

```
# SOLUTION 4.1
from scipy import stats
np.random.seed(42)

group_A = np.random.normal(loc=75, scale=10, size=50)
group_B = np.random.normal(loc=70, scale=12, size=50)

for name, group in [('Group A', group_A), ('Group B', group_B)]:
    print(f'{name}: mean={group.mean():.2f}, std={group.std():.2f}, '
          f'skew={stats.skew(group):.3f}, kurtosis={stats.kurtosis(group):.3f}')

plt.figure(figsize=(8, 4))
plt.hist(group_A, bins=20, alpha=0.6, label='Group A (new method)', color='steelblu
plt.hist(group_B, bins=20, alpha=0.6, label='Group B (old method)', color='coral')
plt.legend()
plt.title('Test Score Distributions')
plt.xlabel('Score')
plt.show()

corr = np.corrcoef(group_A, group_B)[0, 1]
print(f'\nCorrelation between groups: {corr:.3f}')

t_stat, p_value = stats.ttest_ind(group_A, group_B)
print(f'T-statistic: {t_stat:.3f}, P-value: {p_value:.4f}')

if p_value < 0.05:
    print('Conclusion: The difference is statistically significant (p < 0.05). '
          'The new teaching method likely produces better results.')
else:
    print('Conclusion: The difference is NOT statistically significant (p >= 0.05).
          'We cannot conclude the new method is better.')
```

Final Self-Check: Are You Ready for Day 1?

Complete all 5 exercises below without looking at solutions or googling syntax.

If you can do all 5 cleanly, you are ready for Day 1.

If 2 or more feel difficult, go back to the relevant section and spend more time there first.

In [ ]:
```python
# SELF-CHECK 1: Python
# Write a function that takes a list of numbers and returns
# mean, median, and standard deviation without using NumPy.

import math

def compute_stats(numbers):
    # your code here
    pass

test = [4, 8, 15, 16, 23, 42]
result = compute_stats(test)
print(result)
# Expected: mean=18.0, median=15.5, std≈13.09
```

In [ ]:
```python
# SELF-CHECK 2: NumPy and Pandas
# Load the following data into a pandas DataFrame.
# Check for missing values.
# Fill missing salary with the column median.
# Create a new column 'salary_normalized' using min-max normalization.
# Print the first 5 rows of the final DataFrame.

import pandas as pd
import numpy as np

data = {
    'name': ['Alice', 'Bob', 'Carol', 'Dave', 'Eve'],
    'age': [28, 35, 42, 31, np.nan],
    'salary': [75000, np.nan, 95000, 62000, 88000],
    'department': ['Eng', 'HR', 'Eng', 'Sales', 'HR']
}

# your code here
```

In [ ]:
```python
# SELF-CHECK 3: Matplotlib
# Create a 1x2 subplot:
# Left: Histogram of 500 random normal values (mean=0, std=1)
# Right: Scatter plot showing the relationship between
#        x = np.linspace(0, 10, 100) and y = 2*x + noise
# Add titles and axis labels to both.

import matplotlib.pyplot as plt
import numpy as np

np.random.seed(42)
# your code here
```

In [ ]:
```python
# SELF-CHECK 4: Linear Algebra
# Create two 3x3 matrices A and B with random integers between 1 and 10.
# Compute:
# - Matrix multiplication of A and B
# - Transpose of A
```

```
# - Dot product of the first row of A with the first column of B
# - Determinant of A using np.linalg.det()

import numpy as np

np.random.seed(42)
# your code here
```

In [ ]:
```
# SELF-CHECK 5: Statistics
# Generate 1000 random values from a normal distribution (mean=50, std=10).
# Compute: mean, median, std, skewness, kurtosis.
# Identify values that are more than 2 standard deviations from the mean.
# Print how many outliers there are and what % of the data they represent.
# Plot the distribution with a vertical line at mean +/- 2 std.

import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

np.random.seed(42)
# your code here
```

You Passed the Self-Check?

Good. Head to Day 1: What is Data in ML

Some Exercises Felt Difficult?

Go back to that section and spend more time with the resources listed there. There is no shortcut here. A shaky foundation means you will hit a wall on Day 10 instead of building something real by Day 42.

Follow the LinkedIn series: [www.linkedin.com/in/vaishnavi-jagtap-065b753b1]

Star the repo: github.com/VaishnaviJagtap18/42-days-aiml-challenge

Share your progress: #42DaysOfAIML

In [ ]:
```

```